

AutoTestDesign 工具架构与分工方案

1. 项目背景

本项目需要开发一个 AI-driven AutoTestDesign 工具，并使用该工具测试一个独立的目标应用。

本组计划如下：

- 目标应用： `simpletodolist`
- 工具开发目录： `Assignment2`
- 工具技术栈： Python + Streamlit
- 团队人数： 3 人，暂用 A、B、C 表示

本工具的主要目标不是直接测试 AutoTestDesign 本身，而是辅助测试人员对 `simpletodolist` 进行需求分析、风险评估、覆盖项识别、测试策略选择、测试用例生成、交互式审查和测试工件导出。

2. 作业要求理解

根据 `Assignment2.pdf` 和老师补充的 `Requirement_Specification_AutoTestDesign_AI_App.md` ，AutoTestDesign 工具需要覆盖以下能力：

编号	功能	本项目中的实现方向
FR 1.0	输入/解析	支持输入纯文本需求、上传 CSV/Excel 需求文件、手动添加需求
FR 1.1	需求结构化	使用轻量 NLP 从需求中提取功能模块、输入字段、数据范围、条件、动作、预期结果
FR 2.0	风险分析与优先级	使用 AI/ML 风险模型或轻量模型对每条需求计算风险分数，并标记 High / Medium / Low
FR 3.0	黑盒测试设计	支持等价类划分、边界值分析、决策表三种核心技术
FR 4.0	白盒/行为建模	为 Todo 状态变化设计状态转换图，并生成 All States 或 All Transitions 测试序列
FR 5.0	测试预言生成	根据测试数据自动生成 Expected Result 草稿，并允许人工修改
FR 6.0	输出与导出	导出需求分析、风险评分、覆盖项、测试用例、测试套件为 JSON/CSV/Excel

编号	功能	本项目中的实现方向
FR 7.0	测试套件优化	按风险优先级和覆盖效率对测试用例排序、去重或最小化
Interactive Review	交互式审查	允许测试人员修改覆盖项、测试策略、测试用例和优先级

非功能性和技术约束需要重点体现：

类型	要求	本项目中的处理方式
性能	100 条需求处理和风险分析尽量 5 秒内完成	在工具中记录处理耗时，并在文档中说明结果
性能	单条需求完整测试用例生成尽量 2 秒内完成	记录生成耗时，避免复杂外部调用阻塞
可用性	UI 清晰，测试用例可追溯	使用 Streamlit 表格和编辑器展示 Requirement ID、Coverage ID、Technique
安全性	基础数据处理安全	本项目使用本地数据，不上传敏感需求
可维护性	模块化，便于替换算法	将 NLP、风险模型、测试生成、导出拆成独立模块
AI/ML	使用常见 AI/ML 库	推荐使用 <code>scikit-learn</code> ；如环境允许可使用 <code>spaCy</code>
可选 API	可接入外部 LLM API	使用 <code>.env</code> 配置多 provider API Key；前端可切换 DeepSeek、阿里云等 provider 和模型；没有 Key 时回退到本地规则和轻量 ML
数据	需要 mock 数据集和历史执行结果	在 <code>data/</code> 中准备需求数据、历史风险数据、测试执行结果样例
文档	需要架构图和用户手册	在后续文档中补充 System Architecture Diagram 和 User Manual

3. 推荐总体架构

建议采用单体 Streamlit 应用 + 分层 Python 模块的结构。Streamlit 负责交互页面，核心逻辑放在独立 Python 模块中，方便测试、复用和分工。

```
Assignment2/
.env.example
app.py
README.md
requirements.txt
AutoTestDesign_Architecture_and_Team_Plan.md
Requirement_Specification_AutoTestDesign_AI_App.md
docs/
  01_AutoTestDesign_Tool_README.md
  02_Risk_Analysis_Report.md
  03_Test_Plan.md
  04_Detailed_Test_Design_and_Execution.md
  05_Prompt_Design.md
  06_Presentation_Outline.md
  07_User_Manual.md
data/
  projects/
    .gitkeep
  mock_todolist_requirements.csv
  mock_historical_results.csv
  sample_test_execution_results.csv
  sample_todolist_requirements.csv # D/E 后续提供正式版本
exports/
  .gitkeep
src/
  ai_client.py
  prompt_templates.py
  requirement_loader.py
  nlp_processor.py
  requirement_parser.py
  risk_analyzer.py
  ml_risk_model.py
  coverage_identifier.py
  test_strategy_selector.py
  test_case_generator.py
  state_modeler.py
  oracle_generator.py
  suite_optimizer.py
  exporter.py
  persistence.py
  performance_tracker.py
  models.py
tests/
  test_ai_client.py
  test_requirement_parser.py
```

```
test_risk_analyzer.py
test_case_generator.py
```

说明：

- `app.py`：Streamlit 主入口，组织页面和用户交互。
- `src/`：工具核心逻辑，不依赖具体页面。
- `docs/`：项目文档、提示词设计、使用说明。
- `data/`：示例输入数据、mock 历史结果和测试执行结果。
- `exports/`：工具运行后导出的测试工件。
- `data/projects/`：本地保存的项目状态 JSON，例如 `simpletodolist_project.json`。
- `tests/`：对 `AutoTestDesign` 内部模块做基础单元测试。
- `nlp_processor.py`：负责需求分词、关键词提取和结构化字段识别。
- `ml_risk_model.py`：负责训练或调用风险模型，输出风险分和优先级。
- `state_modeler.py`：负责 `Todo` 状态转换建模和测试序列生成。
- `performance_tracker.py`：负责记录需求分析和测试用例生成耗时。
- `persistence.py`：负责保存/加载本地项目 JSON。
- `ai_client.py`：可选 LLM API 调用封装，支持多 provider 和模型切换；没有 `.env` 时不影响本地功能。
- `prompt_templates.py`：集中保存需求结构化、风险解释、覆盖改进等提示词。
- `mock_todolist_requirements.csv`：D/E 正式需求完成前的临时开发数据。
- `sample_todolist_requirements.csv`：D/E 后续提供的正式 `ToDoList` 需求输入文件，当前可以暂时不存在。
- 当前已生成基础脚手架，后续按 A/B/C 分工逐步补充真实逻辑。

4. Streamlit 页面设计

建议将工具界面设计成 6 个主要页面或步骤。

4.1 Requirement Input

目标：完成 FR 1.0。

功能：

- 输入 `simpletodolist` 的需求文本。
- 上传 CSV/Excel 文件。
- 支持手动新增、删除、编辑需求。
- 给每条需求生成唯一 ID，例如 `REQ-TODO-001`。

示例需求：

- 用户可以新增一个 Todo 项。
- 用户可以将 Todo 标记为完成。
- 用户可以删除 Todo。
- 用户不应创建空内容 Todo。
- 系统应保存 Todo 列表并在页面刷新后保持状态。

4.2 Requirement Structuring

目标：完成 FR 1.1。

功能：

- 使用轻量 NLP 或关键词/正则 pipeline 对需求文本进行分词、关键词识别和字段提取。
- 识别每条需求的输入字段。
- 识别有效范围、无效范围、条件、动作、预期结果。
- 将原始需求转化为结构化表格。
- 支持人工修改解析结果。

对 `simpletodolist` 来说，常见结构化字段包括：

- 功能模块：Todo 创建、Todo 编辑、Todo 删除、Todo 完成状态、列表展示。
- 输入字段：Todo 文本、Todo ID、状态值。
- 条件：文本为空、文本超长、Todo 存在、Todo 不存在。
- 期望动作：创建成功、显示错误、状态更新、数据删除。

4.3 Risk Analysis

目标：完成 FR 2.0。

功能：

- 使用 `scikit-learn` 和 mock 历史数据训练轻量风险模型。
- 根据需求文本、影响程度、发生概率、复杂度、用户可见性预测风险等级。
- 风险分映射为 High / Medium / Low。
- 允许测试人员手动调整风险等级，并记录调整理由。

建议采用 ML 模型 + 规则评分 fallback。规则评分可以作为解释依据：

$$\text{Risk Score} = \text{Impact} * 0.4 + \text{Probability} * 0.3 + \text{Complexity} * 0.2 + \text{User Visibility} * 0.1$$

其中每个维度可按 1 到 5 分打分。

4.4 Coverage Item Identification

目标：对应作业中 “Concept → Coverage Item Identification”。

功能：

- 根据结构化需求识别覆盖项。
- 覆盖项需要能追溯到需求 ID。
- 支持测试人员新增、修改、删除覆盖项。

示例覆盖项：

Coverage ID	来源需求	覆盖项
COV-001	REQ-TODO-001	输入合法 Todo 文本时应创建成功
COV-002	REQ-TODO-001	输入空 Todo 文本时应拒绝创建
COV-003	REQ-TODO-002	未完成 Todo 可以切换为完成
COV-004	REQ-TODO-003	已存在 Todo 可以被删除

4.5 Coverage Strategy & Method

目标：对应 FR 3.0 和作业中的 “Coverage Strategy & Method”。

功能：

- 为每个覆盖项选择测试技术。
- 至少支持三种黑盒测试技术：
 - Equivalence Partitioning
 - Boundary Value Analysis
 - Decision Table
- 支持状态转换测试，用于覆盖 Todo 创建、完成、删除等状态变化。
- 支持测试人员手动调整策略。

对 Todo List 的建议策略：

模块	推荐测试技术
Todo 创建	等价类划分、边界值分析
Todo 删除	等价类划分、决策表
Todo 状态切换	状态转换测试

模块	推荐测试技术
Todo 列表展示	等价类划分
输入校验	边界值分析、等价类划分

4.6 Test Case Generation & Review

目标：生成测试用例并体现交互式审查能力。

功能：

- 根据覆盖项和策略生成测试用例。
- 每个测试用例需要包含：
 - Test Case ID
 - Requirement ID
 - Coverage ID
 - Test Technique
 - Precondition
 - Test Data
 - Steps
 - Expected Result
 - Priority
 - Risk Score
- 支持人工修改测试步骤、测试数据、预期结果和优先级。
- 使用模板或规则生成 Expected Result 草稿，体现 FR 5.0。
- 按风险分和覆盖效率对测试用例排序，体现 FR 7.0。
- 修改后保留 traceability。

4.7 Persistence, Export & Result Analysis

目标：完成 FR 6.0，并支持后续报告写作。

功能：

- 保存完整项目状态到 `data/projects/`，例如 `simpletodolist_project.json`。
- 从本地项目 JSON 恢复需求、模型选择和生成结果。
- 导出需求结构化结果。
- 导出风险分析报告数据。
- 导出覆盖项与测试策略映射。
- 导出测试用例。
- 导出测试套件。

- 导出性能记录和可追溯矩阵。
- 可选导出 PyTest/Selenium 脚本草稿。测试执行框架的正式选择主要写入测试计划和详细测试执行文档，不要求工具内置执行能力。

建议导出格式：

- requirements_structured.csv
- risk_analysis.xlsx
- coverage_items.csv
- test_cases.xlsx
- test_suite.json
- traceability_matrix.xlsx
- performance_report.csv

5. 核心数据流

建议工具的数据流如下：

原始需求输入

- > 需求结构化
- > 风险评分
- > 覆盖项识别
- > 覆盖策略选择
- > 测试用例生成
- > 人工交互式审查
- > 测试套件优化
- > 性能记录
- > 导出测试工件
- > 用于测试 simpletodolist

这条流程正好对应作业中强调的：

Concept

- > Coverage Item Identification
- > Coverage Strategy & Method
- > Test Cases and Traceability
- > Prompt Design
- > Results Analysis
- > Improvement with Evidence

6. Prompt 设计思路

如果工具中体现 AI-driven，可以把 AI 能力设计为“提示词模板 + 生成建议 + 人工确认”的形式。

由于 Assignment1 已经积累了 LLM 测试生成、prompt 模式、闭环补充和输出容错经验，Assignment2 中建议把 prompt 和 API 交互统一放到 A 的平台职责中。A 负责维护 prompt 结构、API 调用方式、输出格式约束和失败回退策略；B/C 提供各自模块的测试规则和字段要求。

建议准备以下 prompt：

Prompt 名称	用途	内容提供者	维护者
Requirement Structuring Prompt	将原始需求转换为结构化字段	B	A
Risk Analysis Prompt	分析需求风险并解释理由	B	A
Coverage Identification Prompt	识别覆盖项	B	A
Test Strategy Prompt	为覆盖项选择测试技术	C	A
Test Case Generation Prompt	生成测试用例	C	A
Oracle Generation Prompt	生成 Expected Result	C	A
Improvement Prompt	根据人工反馈补充遗漏覆盖项和测试用例	B/C	A

为了避免完全依赖大模型，建议采用规则引擎 + prompt 辅助的方式：

- scikit-learn 负责轻量 AI/ML 风险预测。
- 关键词、正则和可编辑表格负责稳定的需求结构化。
- 可选 LLM API 负责生成解释、补充覆盖建议和改进建议；前端可选择 provider 和模型。
- .env.example 提交到仓库，真实 .env 不提交。
- 规则引擎保证稳定输出。
- Prompt 用于生成解释、补充覆盖项、优化测试用例。
- 所有 AI 输出都必须经过测试人员交互式确认。

A 在 prompt/API 方面主要负责：

- 统一 prompt 模板风格和输出格式。
- 将 B/C 提供的领域规则整理成可复用 prompt。
- 维护 src/prompt_templates.py 。
- 维护 docs/05_Prompt_Design.md 。
- 设计 LLM 输出失败时的回退策略，例如解析失败时退回本地规则结果。
- 记录 prompt 版本、使用场景和人工审查结论。

B/C 的职责不是各自随意写 prompt，而是提供本模块的测试知识：

- B 提供需求结构化、风险分析、覆盖项识别需要的字段和判断规则。
- C 提供测试策略、测试用例、Expected Result、优化建议需要的字段和判断规则。
- A 把这些规则整理成统一 prompt，并保证格式稳定、可解析、可审查。

7. 针对 simpletodolist 的测试范围建议

建议选择 simpletodolist 的 Todo 管理功能作为主要测试对象。

优先测试模块：

1. Todo 创建
2. Todo 删除
3. Todo 完成状态切换
4. Todo 列表展示
5. 输入校验
6. 数据持久化或页面刷新后的状态保持，如果目标应用支持

详细测试设计文档可以聚焦一个主要模块。建议选择“Todo 创建模块”，因为它容易展示：

- 等价类划分
- 边界值分析
- 决策表
- 风险分析
- 测试数据设计
- 自动化测试脚本

数据集说明：

- simpletodolist 是被测试的目标应用，不是数据集。
- sample_todolist_requirements.csv 是 TodoList 的需求输入数据，由负责 TodoList 需求整理的同学提供，用于驱动 AutoTestDesign 生成测试设计。
- mock_historical_results.csv 是模拟历史需求和缺陷结果，用于训练或演示风险分析模型。
- sample_test_execution_results.csv 是测试执行结果样例，用于后续结果分析和报告展示。

8. 三人分工方案

分工不要按“谁负责哪个 FR”来切，否则会在覆盖项、测试策略和测试用例之间产生交叉。建议按工具流水线切成三段：

- A: 平台和界面
- B: 需求理解和风险分析
- C: 测试设计和导出

三个人只通过明确的数据结构交接，不互相修改对方模块的内部逻辑。

A: 平台、界面与集成负责人

A 不负责具体测试算法，主要负责让工具能被使用、能演示、能集成 B/C 的结果。

负责范围：

- 设计 AutoTestDesign 总体页面流程。
- 搭建 Streamlit 应用框架。
- 负责需求输入、页面导航、会话状态管理。
- 集成 B 的需求分析结果和 C 的测试设计结果。
- 增加性能展示区域，显示需求分析耗时和测试用例生成耗时。
- 维护可选 LLM API 调用、prompt 模板和输出失败回退策略。
- 维护 README.md、System Architecture Diagram 和 User Manual。
- 负责最终演示视频中的工具主流程展示。

输入：

- B 输出的结构化需求、风险结果、覆盖项。
- C 输出的测试策略、测试用例、导出文件。

输出：

- 可运行的 Streamlit 工具。
- 用户能操作的交互界面。
- README、用户手册、架构图、演示流程。

不负责：

- 不设计风险模型。
- 不设计 EP/BVA/Decision Table 生成规则。
- 不直接改 B/C 的核心算法。
- 不单独决定 B/C 模块的测试规则，只负责把 B/C 提供的规则整理成稳定 prompt。

建议负责文件：

- app.py
- README.md

- requirements.txt
- .env.example
- src/ai_client.py
- src/prompt_templates.py
- src/performance_tracker.py
- src/persistence.py
- docs/01_AutoTestDesign_Tool_README.md
- docs/05_Prompt_Design.md
- docs/07_User_Manual.md
- System Architecture Diagram

B：需求理解、风险分析与覆盖项负责人

B 负责把已整理好的需求输入变成“测试设计可以使用的结构化数据”。B 的起点是 D/E 同学提供的 TodoList 需求输入文件，终点是覆盖项，不进入具体测试用例生成。

负责范围：

- 接收 D/E 同学整理好的 simpletodolist 需求输入样例。在D/E 同学完成之前 可以先用 mock 数据开发和测试。
- 检查需求输入格式是否能被 AutoTestDesign 工具读取。
- 负责 NLP 需求解析和结构化方案。
- 设计 scikit-learn 风险评分模型。
- 准备 mock 历史数据集。
- 设计覆盖项识别逻辑。
- 输出风险分析报告所需数据。
- 编写风险分析报告初稿。

输入：

- D/E 同学提供的原始需求文本。
- D/E 同学提供的 CSV/Excel 需求文件。
- mock 历史风险数据。

输出：

- Requirement：结构化需求。
- RiskRecord：风险分数和优先级。
- CoverageItem：覆盖项。
- 风险分析报告数据。

不负责：

- 不生成最终测试用例步骤。
- 不决定具体导出格式。
- 不写 Streamlit 页面集成逻辑。

对应作业内容：

- FR 1.0
- FR 1.1
- FR 2.0
- Coverage Item Identification
- Risk Analysis Report

建议负责文件：

- `src/requirement_loader.py`
- `src/nlp_processor.py`
- `src/requirement_parser.py`
- `src/risk_analyzer.py`
- `src/ml_risk_model.py`
- `src/coverage_identifier.py`
- 不负责维护 `data/sample_todolist_requirements.csv` 的业务内容，只负责读取格式和字段约定。
- `data/mock_historical_results.csv`
- 风险分析报告相关文档

C：测试设计、测试用例生成与导出负责人

C 负责从 B 的覆盖项开始，生成测试策略、测试用例、测试套件和导出结果。C 不重新解析需求，也不重算风险。

负责范围：

- 设计黑盒测试用例生成逻辑。
- 实现等价类划分、边界值分析、决策表的生成规则。
- 设计 Todo 状态转换测试。
- 实现 Expected Result 草稿生成。
- 实现测试套件排序、去重或最小化。
- 设计测试用例字段和可追溯矩阵。
- 负责导出 CSV/JSON/Excel。
- 在测试计划和详细测试设计文档中说明测试执行框架选择；工具内可选提供 Selenium/PyTest 脚本草稿。
- 编写详细测试设计与执行文档初稿。

输入：

- B 输出的 Requirement 、 RiskRecord 、 CoverageItem 。

输出：

- TestStrategy ： 每个覆盖项对应的测试技术。
- TestCase ： 完整测试用例。
- TraceabilityRecord ： 需求、覆盖项、策略、测试用例之间的映射。
- 导出文件：CSV、JSON、Excel。
- 详细测试设计与执行文档数据。

不负责：

- 不修改原始需求解析逻辑。
- 不训练风险模型。
- 不负责主页面框架。

对应作业内容：

- FR 3.0
- FR 4.0
- FR 5.0
- FR 6.0
- FR 7.0
- Test Cases and Traceability
- Detailed Test Design and Execution Document
- Test Tool Implementation

建议负责文件：

- src/test_strategy_selector.py
- src/test_case_generator.py
- src/state_modeler.py
- src/oracle_generator.py
- src/suite_optimizer.py
- src/exporter.py
- 详细测试设计与执行文档

9. 协作边界

为了减少冲突，三个人按数据流交接，不按同一个功能页面混写。

阶段	负责人	输入	输出	交给谁
1. 页面和流程	A	用户操作	原始输入、按钮事件、展示页面	B / C
2. 需求理解	B	原始需求	Requirement	A / C
3. 风险分析	B	Requirement + mock 历史数据	RiskRecord	A / C
4. 覆盖项识别	B	Requirement + RiskRecord	CoverageItem	A / C
5. 测试策略选择	C	CoverageItem	TestStrategy	A
6. 测试用例生成	C	Requirement + CoverageItem + TestStrategy	TestCase	A
7. 导出与追溯矩阵	C	TestCase + TraceabilityRecord	CSV/JSON/Excel	A
8. 展示、 手动修改、演示	A	B/C 的输出	可交互工具和演示材料	全组

交叉点的处理规则：

- 覆盖项归 B，因为覆盖项来自需求理解。
- 测试策略归 C，因为策略决定如何生成测试用例。
- 风险等级归 B，但 C 可以读取风险等级来排序测试用例。
- 导出格式归 C，但导出按钮和下载界面归 A。
- 人工修改界面归 A，但修改后的数据仍按所属模块回写，例如风险回写给 B 的数据结构，测试用例回写给 C 的数据结构。

具体文件责任表：

文件/目录	负责人	说明
app.py	A	Streamlit 主入口、页面流程、按钮、表格展示、集成 B/C 输出
requirements.txt	A	项目依赖，B/C 如需新增库先和 A 对齐

文件/目录	负责人	说明
README.md	A	工具安装、运行方式、项目结构说明
docs/01_AutoTestDesign_Tool_README.md	A	工具说明文档
docs/07_User_Manual.md	A	用户手册
src/performance_tracker.py	A	记录和展示性能耗时
src/ai_client.py	A	可选 LLM API 配置和调用封装
src/prompt_templates.py	A	统一维护 prompt 模板；B/C 提供各自模块的规则和字段要求
src/persistence.py	A	本地项目 JSON 保存和加载
src/requirement_loader.py	B	读取 D/E 提供的需求文件或 mock 需求文件
src/nlp_processor.py	B	轻量 NLP、关键词识别、需求字段抽取
src/requirement_parser.py	B	输出结构化需求 Requirement
src/risk_analyzer.py	B	风险评分流程
src/ml_risk_model.py	B	scikit-learn 风险模型或轻量预测逻辑
src/coverage_identifier.py	B	根据结构化需求识别 CoverageItem
data/mock_todolist_requirements.csv	B	D/E 正式需求完成前的临时开发数据
data/mock_historical_results.csv	B	风险模型训练/演示用 mock 历史数据
docs/02_Risk_Analysis_Report.md	B	目标应用风险分析报告
src/test_strategy_selector.py	C	覆盖项到测试技术的选择
src/test_case_generator.py	C	生成测试用例
src/state_modeler.py	C	Todo 状态转换模型
src/oracle_generator.py	C	Expected Result 草稿生成

文件/目录	负责人	说明
src/suite_optimizer.py	C	测试套件排序、去重、最小化
src/exporter.py	C	CSV/JSON/Excel 导出逻辑
docs/03_Test_Plan.md	C 为主，A/B 补充	测试计划， 包含执行框架选择说明
docs/04_Detailed_Test_Design_and_Execution.md	C	详细测试设计与执行文档
docs/05_Prompt_Design.md	A	记录 prompt 模板、版本、用途、 输出格式和人工审查策略；B/C 补充模块规则
docs/06_Presentation_Outline.md	A 为主， 全组补充	演示结构和分工
tests/	对应模块负责人	谁负责模块，谁补对应测试

建议统一数据结构：

Requirement
CoverageItem
RiskRecord
TestStrategy
TestCase
TraceabilityRecord
PerformanceRecord

这些结构可以后续在 `src/models.py` 中统一定义。

10. 文档交付建议

最终至少需要准备以下文档：

```
Assignment2/  
docs/  
  01_AutoTestDesign_Tool_README.md  
  02_Risk_Analysis_Report.md  
  03_Test_Plan.md  
  04_Detailed_Test_Design_and_Execution.md  
  05_Prompt_Design.md  
  06_Presentation_Outline.md  
  07_User_Manual.md
```

最终提交时，报告和 PPT 需要转为 PDF，工具材料需要压缩提交。

11. 推荐实施顺序

建议按以下顺序推进：

1. 明确 `simpletodolist` 的功能需求。
2. 编写 `ToDoList` 需求数据、mock 历史风险数据和测试执行结果样例。
3. 搭建 `Streamlit` 页面骨架。
4. 完成 `NLP` 需求结构化和 `AI/ML` 风险评分。
5. 完成覆盖项识别和交互式修改。
6. 完成三种黑盒测试技术的用例生成。
7. 完成状态转换测试、`Expected Result` 生成和测试套件优化的轻量版本。
8. 完成导出功能和性能记录。
9. 对 `ToDo` 创建模块做详细测试设计。
10. 在测试计划和详细测试设计文档中说明 `Selenium/PyTest` 的选择，并可手动执行部分测试脚本。
11. 写风险报告、测试计划、详细测试设计文档和用户手册。
12. 准备 PPT 和演示视频。

12. 建议优先级

必须优先完成：

- `Streamlit` 工具主流程
- 需求导入
- `NLP` 需求结构化
- `AI/ML` 风险评分
- 覆盖项识别
- 三种黑盒测试技术
- 状态转换测试

- Expected Result 生成
- 测试套件优化
- 测试用例生成
- 人工交互式修改
- 性能记录
- CSV/Excel/JSON 导出

时间允许再完成：

- Selenium/PyTest 脚本自动生成
- 更复杂的 NLP 模型或外部大模型接入

13. 推荐结论

本组最合适的方案是：

`Python + Streamlit + Pandas + OpenPyXL + scikit-learn`

如环境允许，可补充 `spaCy`；如果安装受限，可以使用 `scikit-learn` 的 `TfidfVectorizer` 加关键词/正则规则实现轻量 NLP。

测试执行框架建议写入测试计划和详细测试执行文档：

`Selenium` 或 `PyTest`

如果 `simpletodolist` 是纯前端页面，优先选择 `Selenium`，因为它更适合演示用户真实操作流程，例如新增 `Todo`、点击完成、删除 `Todo`。

如果后续重构或封装了可测试函数，则可以用 `PyTest` 做更轻量的单元测试。

整体策略应以“完整流程可演示”为优先目标，而不是追求复杂 AI 算法。老师规约确认后，本项目应定位为“轻量 AI/ML + 规则生成 + 人工交互式审查”的 `AutoTestDesign` 工具。评分重点在于概念理解、设计一致性、覆盖有效性、深入分析和展示，因此工具需要清楚体现：

- 人类测试设计者参与
- 覆盖项可审查
- 策略可修改
- 测试用例可追溯
- 结果可导出
- AI/ML 风险分析有 mock 数据支撑
- 性能目标有记录和说明

- 对 `simpletodolist` 的测试过程有证据链